Graphical User Interface of the SOFIA Vision System

Marco Heimdal Castro Magaña, Alejandro Amador Franco, Karen Alejandra Guevara Vasquez, María Alicia Vela Contreras, Juan Pablo Contreras Quiñones, Jorge A. Lizarraga, Luis E. González-Jiménez, Luis F. Luque-Vega

***Abstract*—SOFIA is a modular kinetic surface that responds to the gestures of a user. This work presents its graphical user interface (GUI): the window that shows the camera, controls the gesture detection, and sends each recognized gesture to the robot or to its simulation. The GUI runs the video processing and each communication link in separate threads, so that it never freezes. Its behavior is explained with three state machines: the processing thread, which can be started, paused, resumed, and stopped; the reliable sending of commands, with acknowledgements and retransmission; and the link supervision. The same commands can be sent to the robot, to a Webots simulation, or to both at the same time, which turns the simulation into a digital twin of the robot.**

***Index Terms*—Graphical user interface, multithreading, state machines, communication protocol, gesture recognition, Webots, digital twin.**

I. SOFIA in Brief

SOFIA is an interactive installation made of 16 hexagonal units in a honeycomb arrangement; each unit can tilt in two directions [1]. It has four parts: (i) a *vision system* that watches the user with a camera and recognizes five gestures (INICIO, RECARGAR, ESCUDO, ATAQUE, and FIN) or follows the position of the right hand, using MediaPipe [2]; (ii) the *GUI*, which shows the result and controls the system; (iii) a *robot controller*, which receives commands through a wireless link and moves the units; and (iv) a *simulation* of the 16 units in Webots [3], which can play the role of the robot. This paper is about the GUI and how it talks to the robot and to the simulation.

II. Architecture

The GUI was built with PySide6 [4], the Python version of Qt, because it provides threads that are integrated with the interface (`QThread`), signals to pass data between threads safely, and a way to pause a thread without consuming CPU. Fig. 1 shows the threads. The *GUI thread* only draws the window and handles the buttons. The *vision thread* reads the camera and runs the detection, which takes tens of milliseconds per frame. Each *link thread* talks to one target, the robot over Bluetooth Low Energy [5] or the simulation over a local TCP socket, because waiting for an answer may take from milliseconds to seconds. A last thread receives the images of a virtual camera of the simulation. Threads never touch the widgets: they send their results to the GUI thread through Qt signals.

The window has the video on the left and, on the right, the controls in the order in which they are used: detection (start, pause, resume, stop), mode, result, targets (robot and simulation), and an event log with every command sent and its answer. There are three modes: *game*, where each new gesture is sent once; *tracking*, where the position of the right hand is sent continuously; and *test*, where the GUI asks the user for gestures and measures how often and how fast they are recognized. The image is shown as a mirror, as in a selfie; since the detector names the hands for a non-mirrored image, the GUI swaps them, so that tracking follows the real right hand and ignores the other one.

The authors are affiliated with the Instituto Tecnológico de Estudios Superiores de Occidente (ITESO), Tlaquepaque, Jalisco, Mexico.

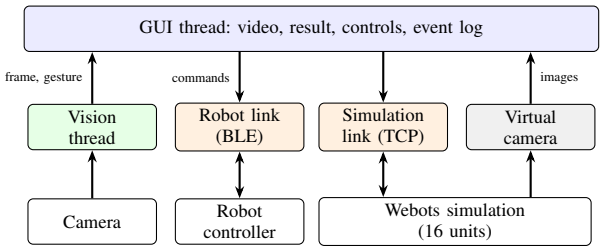


Fig. 1. Threads of the GUI. Each box below the GUI thread runs in its own thread and only communicates with it through Qt signals.

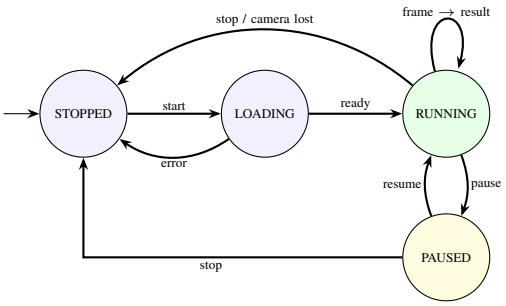


Fig. 2. State machine of the processing thread.

III. The Processing Thread

The vision thread follows the state machine of Fig. 2. *Start* creates the thread, which opens the camera and loads the models (LOADING) and then processes frames (RUNNING). *Pause* makes the thread wait on a condition without using the CPU, with the camera still open; *resume* wakes it up. *Stop* ends the loop, releases the camera, and sends a STOP command to the robot. Any error, such as a camera that cannot be opened or that stops sending frames, also leads to STOPPED with a message in the window, instead of closing the application. The buttons are enabled according to the state, so only valid transitions can be requested.

TABLE I
MESSAGES

Type	Direction	Meaning
POSE	GUI → robot	Recognized gesture (1–5)
TRACK	GUI → robot	Hand position, no acknowledgement
MODE	GUI → robot	Active mode (game, test, tracking)
PING	GUI → robot	Test of the link and heartbeat
STOP	GUI → robot	Go to a safe position
ACK / NACK	robot → GUI	Received / rejected, with an error code

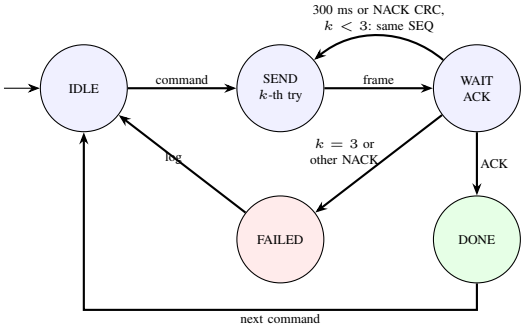


Fig. 3. State machine of the reliable sending of a command (stop-and-wait with retransmission).

IV. Talking to the Robot

A. Messages

Each message is a short frame: a start byte (`0xA5`), the type, a sequence number, the payload length, the payload (0–14 bytes), and a CRC-8 checksum that detects damaged frames. The messages are listed in Table I. A gesture is sent with POSE only when it changes, and the hand position with TRACK up to 20 times per second; TRACK is not acknowledged, since a newer position replaces an older one.

B. Reliable Sending

Every command except TRACK is sent with the state machine of Fig. 3, one at a time. After sending, the link waits up to 300 ms for an acknowledgement that matches the type and sequence number of the command. If it does not arrive, or if the frame arrived damaged (NACK with a CRC error), the command is sent again *with the same sequence number*, up to three times; if all fail, it is reported as failed in the event log. Since the sequence number does not change, the robot recognizes a repeated command whose first acknowledgement was lost, answers it again, and does not repeat the movement.

C. Link Supervision

The link itself follows the state machine of Fig. 4. When the user presses *connect*, the GUI searches for the robot and opens the link; on success, it sends the active mode. While connected, if nothing has been sent for 1 s, the GUI sends a PING (heartbeat), so that the robot can tell a quiet link from a lost one: if the robot receives nothing for 3 s, it moves to a safe position by itself. If the link is lost, the GUI shows it in the window and returns to DISCONNECTED without closing.

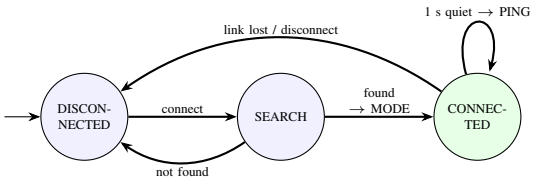


Fig. 4. State machine of the link supervision.

V. Simulation and Digital Twin

The simulation receives *exactly the same frames* as the robot, so it also tests the protocol, not only the movements. Since TCP carries a stream of bytes instead of separate packets, each frame is preceded by one byte with its length. A simulated controller in Webots validates and acknowledges the frames with the same rules and moves the 16 simulated units: in game mode, each gesture draws a pattern with the whole array; in tracking mode, the units point at a sphere that follows the hand. A local link never loses frames, so the GUI can optionally add delay (30 ± 10 ms), losses (3 %), and damaged bytes (2 %), like a wireless link, to see the retransmissions of Fig. 3 in action. A virtual camera above the array sends its images (320×320 px, about 15 per second) through a separate socket and thread, so the user sees, in the same window, the gesture and the response of the simulated array.

The GUI sends each command to *every* connected target. With only the simulation connected, the whole system can be tested without the robot; with only the robot, it works as usual; and with both, every command reaches the robot and the simulation at the same time, which then works as a digital twin. The event log labels each answer with its target, so both responses can be compared.

VI. Verification

The GUI was tested with the camera and the detection replaced by stubs, applying the controls in sequence: the thread went through LOADING, RUNNING, PAUSED, RUNNING, and STOPPED, and sent no frames while paused. The link was tested against a simulated robot: commands were acknowledged; a lost acknowledgement caused a retransmission with the same sequence number and a single execution; a robot that never answers produced three attempts and a failure; TRACK was limited to 20 frames per second; and a PING was sent after 1 s without traffic. With the Webots simulation, every gesture drew its pattern on the 16 units, and the units followed the sphere only in tracking mode. A preliminary test with the physical robot confirmed the wireless link and the movement of one unit. Pending are a quantitative measurement on the robot and the test of the digital twin with both targets connected.

VII. Conclusions

The GUI of SOFIA keeps the interface responsive by running the video processing and each communication link in separate threads. Its main behaviors are simple state machines: the processing thread (start, pause, resume, stop, and recovery from camera errors), the reliable sending of commands

(acknowledgement, retransmission with the same sequence number, and failure report), and the link supervision (heartbeat and safe position). Because the simulation speaks the same protocol as the robot, the system can be developed and tested without hardware and can run a digital twin of the robot. Future work includes the automatic reconnection of the link, the stress tests with several users and lighting conditions, and the measurement of the time from gesture to movement on the robot.

REFERENCES

[1] J. A. Lizárraga, "Presentación del labdst: Sesión 1 - contexto, metodología d-lab y sofia," 2026, iTESO, 17 de agosto de 2026.
[2] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A framework for building perception pipelines," *arXiv preprint arXiv:1906.08172*, 2019.
[3] O. Michel, "Cyberbotics ltd. Webots: Professional mobile robot simulation," *International Journal of Advanced Robotic Systems*, vol. 1, no. 1, pp. 39–42, 2004.
[4] The Qt Company, "Qt for Python (PySide6) documentation," 2026. [Online]. Available: https://doc.qt.io/qtforpython-6/
[5] H. Blidh, "Bleak: Bluetooth low energy platform agnostic klient," 2026. [Online]. Available: https://github.com/hbldh/bleak